PRODUCTION-STYLE PLATFORM LAB

From commit to observable rollback

A reviewable GitOps delivery path for a multi-tenant Kubernetes service.

The artifact is immutable. Desired state is reviewed. The cluster reconciles. Prometheus decides whether the release progresses or rolls back.

BUILT AND DOCUMENTED BY

Syed Mohammad Shah Mostafa (Tash)

Platform & Backend Engineer building observable AI systems | Paris

OPEN CASE STUDY

INSPECT REPOSITORY

VIEW GITHUB PROFILE

The delivery path

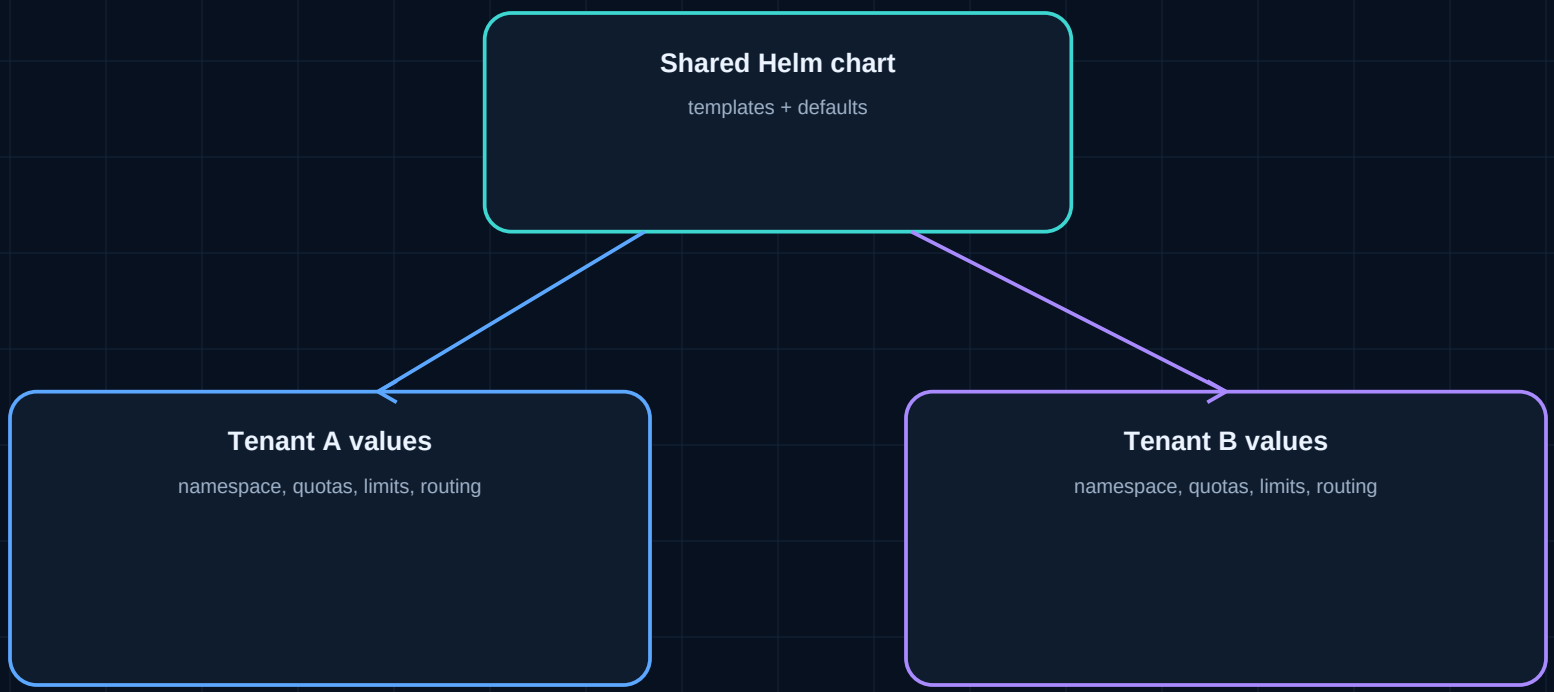
CI produces evidence and an artifact. Git carries the reviewed intent. Argo CD owns reconciliation inside the cluster.

Boundary that matters

The workflow does not use CI credentials to push arbitrary runtime changes into the cluster. CI updates versioned intent; the reconciler applies it.

Tenant differences stay reviewable

One chart carries shared behavior. Per-tenant values express controlled differences without copying entire manifest trees.



RBAC

service identity and permissions

NetworkPolicy

profile-dependent traffic boundaries

Quota

namespace resource ceilings

Limits

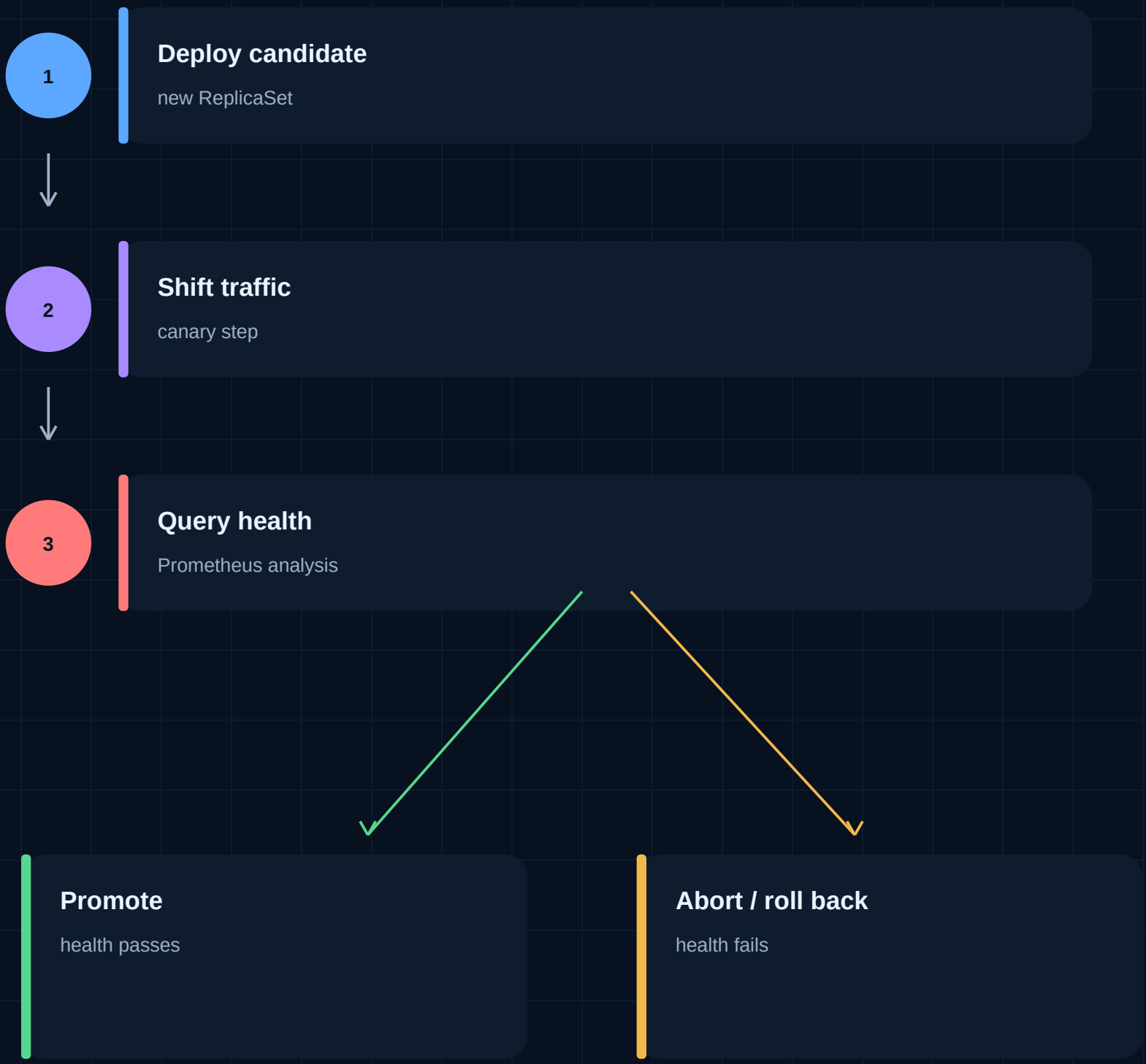
container requests and caps

What this proves

- tenant configuration can be reviewed as data
- shared templates reduce drift between tenant deployments
- security and resource controls are visible in the rendered manifests

A release should be able to fail safely

The rollout controller changes traffic in steps. Prometheus supplies a measurable health signal before promotion.



OPERATIONAL PRINCIPLE

Telemetry is part of the release decision, not a dashboard added after deployment.

The tool list is not the argument

The useful signal is where responsibilities stop, how failures are handled, and which claims can be reproduced.

CI builds; Git declares

- publish immutable artifacts
- update reviewed desired state
- leave cluster reconciliation to Argo CD

Configuration is data

- share templates
- keep tenant differences explicit
- render and lint before deployment

Health has a failure path

- query Prometheus during rollout
- promote only when analysis passes
- make abort behavior inspectable

Evidence accompanies claims

- source paths and workflows
- tests and verification commands
- limitations published beside outcomes

Trade-offs

- A local or single-cluster lab makes the control flow reproducible, but it does not prove multi-cluster operations.
- Prometheus analysis demonstrates measurable release gates, but synthetic or lab traffic cannot stand in for production SLO history.
- Git review improves traceability, but it also adds a repository update step and requires careful secret boundaries.

What you can inspect

This brief is an entry point. The repository contains the implementation and the case study explains the reasoning.

Verification paths

- Python tests and Ruff
- Helm lint for both tenants
- workflow and manifest review
- reproducible quick-start commands

Public evidence

- Argo Rollout templates
- Prometheus analysis configuration
- RBAC, NetworkPolicy, quotas, limits
- GitHub Actions and GHCR path

What this does not claim

- No claim of production traffic, enterprise scale, or EKS deployment.
- No claim of reduced MTTR or real on-call incident ownership.
- No claim that a production-style lab replaces operating a live platform.
- The project demonstrates design choices, implementation, and verification within its stated scope.

CONTINUE THE REVIEW

READ THE FULL CASE STUDY

INSPECT SOURCE AND TESTS

Platform & Backend Engineer building observable AI systems

Syed Mohammad Shah Mostafa (Tash) | Paris, France